

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Information and Communication Technology Department



BACHELOR THESIS

Tran Hien Chuong

Student ID: 22BA13056

Supervised by:

Dr. Tran Hoang Tung

Title:

**AI Assistant for Vietnamese Labor Law: A Case Study
on Employment Contract Termination**

Program: Information and Communication Technology

Academic Year: 2025–2026

To whom it may concern,

I, Dr. Tran Hoang Tung, certify that the thesis report of Mr. Tran Hien Chuong is qualified to be presented in the Thesis Jury 2025–2026.

Hanoi, July 1, 2026

Supervisor's signature

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iii
LIST OF ABBREVIATIONS	iv
LIST OF TABLES	v
LIST OF FIGURES	vi
ABSTRACT	vii
1 Introduction	1
1.1 Context and Problem Statement	1
1.2 Objectives and Scope	1
1.3 Contributions and Thesis Organization	2
2 Background and Related Work	2
2.1 Retrieval-Augmented Generation	2
2.2 Legal Question Answering	3
2.3 Hybrid Retrieval for Legal Evidence	3
2.4 Graph-Augmented Retrieval	3
2.5 Evaluation Requirements for Legal QA	4
2.6 Research Gap	4
3 System Design and Methodology	4
3.1 Overall RAG Pipeline	4
3.2 Scoped Legal Corpus	5
3.3 Hierarchy-Aware Legal Processing	6
3.4 Hybrid Retrieval Pipeline	7
3.5 Policy-Controlled Legal Graph Expansion	7
3.6 Context Assembly and Budgeting	8
3.7 Citation Validation and Insufficient-Context Handling	8
4 Implementation and Deployment	9
4.1 Technology Stack	9
4.2 Offline Corpus, Index, and Graph Build	10
4.3 Backend Runtime and Chat Flow	10
4.4 Answer Generation and Safety	11
4.5 Frontend and Deployment Architecture	11
4.6 User Interface Demonstration	11
4.7 Evaluation Reproducibility	12
5 Evaluation and Discussion	13
5.1 Benchmark Design and Evaluation Metrics	13
5.2 Retrieval Results	14
5.3 End-to-End Results	15
5.4 Category-Level Results	16
5.5 Case Studies and Error Taxonomy	16
5.6 Discussion	17
5.7 Threats to Validity	18
6 Conclusion and Future Work	18
6.1 Conclusion	18
6.2 Future Work	19
6.3 Final Remark	19
References	20
A Example Legal Evidence Chunk	22
B Example Legal Graph	23
C Benchmark Design Details	24
D Additional Evaluation Examples	25

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Dr. Tran Hoang Tung, for his guidance, feedback, and support throughout the development of this thesis. His advice helped me shape the research problem, refine the system design, and present the work more clearly.

I am also grateful to the lecturers and staff of the Information and Communication Technology Department at the University of Science and Technology of Hanoi for providing the academic foundation and learning environment that made this project possible.

Finally, I would like to thank my family and friends for their encouragement, patience, and support during the completion of this thesis.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BLLD	Vietnamese Labor Code 2019
BLTTDS	Vietnamese Civil Procedure Code 2015
E2E	End-to-End
LLM	Large Language Model
QA	Question Answering
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SBERT	Sentence-BERT
UI	User Interface

LIST OF TABLES

1	Scoped Legal Corpus Summary	6
2	Structure of a Legal Evidence Chunk	7
3	Technology Stack and System Modules	9
4	Evaluation Metrics Summary	14
5	Benchmark Composition	14
6	Retrieval and Graph Expansion Results	15
7	End-to-End QA and Citation Validation Results	15
8	Case Studies and Error Taxonomy	17
9	Benchmark category design.	24
10	Representative final evaluation cases.	25

LIST OF FIGURES

1	RAG pipeline for citation-grounded Vietnamese labor-law question answering.	5
2	System architecture and deployment components of the Vietnamese Labor Law AI Assistant.	10
3	Example in-corpus legal question answered with cited legal basis.	12
4	Insufficient-context response for an unsupported or out-of-corpus legal question.	12
5	Example legal graph expansion for retirement-age evidence.	23

ABSTRACT

This thesis presents an AI assistant for Vietnamese labor-law questions, with a case-study focus on employment contract termination and related employment issues. The system is designed as a citation-grounded legal information assistant over a scoped corpus that includes selected labor-law documents, decrees, circulars, and labor-related civil procedure provisions. To improve legal grounding, the project preserves legal structure during corpus processing, represents relevant provisions in Qdrant and Neo4j, retrieves supporting evidence, generates answers from that evidence, and validates whether the final citations are present in the retrieved context. The implemented assistant uses a FastAPI backend, a React/Vite frontend, Supabase, Qdrant, Neo4j, and a configured language-model provider. The system is evaluated on a fixed benchmark of 100 questions, including both in-corpus and out-of-corpus cases, to measure retrieval performance, answer quality, citation validation, and refusal behavior. The results show that graph-augmented retrieval and citation validation can strengthen legal evidence control, while retrieval completeness remains the main limitation. The assistant is intended to support access to legal information within the indexed corpus and does not replace professional legal advice.

Keywords: Vietnamese labor law; Legal question answering; Retrieval-Augmented Generation; Hybrid retrieval; Legal graph; Citation validation.

1 Introduction

1.1 Context and Problem Statement

Legal information is difficult to use when it is scattered across many documents. In Vietnamese labor law, a practical question is often not answered by one paragraph alone. A user may ask about the termination of an employment contract, severance allowance, retirement age, working time, minor workers, or labor disputes. To answer correctly, the system may need to find rules from the Labor Code, a decree, a circular, or even a table or appendix.

This difficulty is increased by the structure of Vietnamese legal documents. Different legal instruments may overlap, refer to each other, provide implementation details, or replace earlier rules [1, 2]. A later document may clarify or supersede a previous one, while a lower-level document may explain how a higher-level legal rule should be applied. Therefore, a legal question-answering system should not treat legal texts as independent passages only. It should also preserve the relationships between legal provisions, documents, and legal ranks.

This creates an important challenge for legal question answering. A useful answer should not only be fluent and easy to read. It must also show the legal basis behind the answer. In law, the citation is part of the information itself. Users need to know which document, article, clause, point, or appendix supports the response.

Recent language models can generate natural answers, but they do not automatically guarantee legal grounding [3]. If the relevant legal text is missing, incomplete, outdated, or incorrectly selected, the final answer may still sound convincing. This is a serious limitation for legal information systems. Therefore, the main problem is not only how to generate an answer, but how to retrieve the right legal evidence before generation and check the cited sources after generation.

This thesis addresses that problem through a Vietnamese labor-law AI assistant. The system is designed for citation-grounded question answering over a scoped legal corpus. It retrieves relevant legal provisions, assembles them as supporting evidence, generates an answer from this evidence, and validates whether the final citations are supported by the retrieved context.

1.2 Objectives and Scope

The objective of this thesis is to design, implement, and evaluate an AI assistant for Vietnamese labor-law questions, with a case-study focus on employment contract termination and related employment issues.

The first objective is to prepare a structured legal corpus. Instead of treating legal documents as ordinary plain text, the system preserves important legal units such as documents, articles, clauses, points, appendices, and legal rank. This structure helps the system keep track of where each rule comes from.

The second objective is to build a retrieval pipeline for legal evidence. The system combines hybrid retrieval in Qdrant with a Neo4j legal graph. Qdrant is used to search for relevant legal text, while Neo4j is used to represent legal relationships such as hierarchy, cross-references, implementation guidance, and connections between related provisions. This graph-based representation is useful because legal relevance often depends on document relationships, not only textual similarity.

The third objective is to connect retrieval with answer generation. The assistant generates answers from selected legal contexts and then checks whether the cited legal bases actually appear in the retrieved evidence. This step is used to reduce unsupported citations.

The fourth objective is to evaluate the system on a fixed benchmark of 100 questions. The benchmark includes both questions covered by the indexed corpus and questions outside the corpus. This allows the evaluation to measure retrieval performance, answer quality, citation validation, and refusal behavior when there is not enough legal evidence.

The scope of the system is intentionally limited. The indexed corpus includes selected Vietnamese labor-law documents: the Labor Code 2019 and the labor-related subset of the Civil Procedure Code 2015 [4, 5], Decree 135/2020/ND-CP and Decree 145/2020/ND-CP [6, 7], and Circular 09/2020/TT-BLDTBXH and Circular 10/2020/TT-BLDTBXH [8, 9]. The system is a legal information assistant for this corpus, not a replacement for professional legal advice.

1.3 Contributions and Thesis Organization

The main contributions of this thesis are implementation-oriented. First, it builds a hierarchy-aware processing pipeline for Vietnamese labor-law documents. Second, it implements a retrieval system that combines Qdrant hybrid search with a Neo4j legal graph. Third, it adds citation validation to check whether generated answers remain grounded in retrieved legal evidence. Fourth, it provides a working web-based assistant using a FastAPI backend, a React/Vite frontend, Supabase, Qdrant, Neo4j, and a configured language-model provider. Finally, it evaluates the system using a reproducible 100-question benchmark.

The rest of this thesis is organized as follows. Section 2 reviews the background and related work on retrieval-augmented generation, legal question answering, graph-augmented retrieval, and evaluation for legal QA systems. Section 3 presents the system design and methodology. Section 4 describes the implementation and deployment. Section 5 reports the evaluation results and discusses the main limitations. Section 6 concludes the thesis and suggests future work.

2 Background and Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a common approach for building question-answering systems over external documents [10]. Instead of asking a language model to answer only from its internal knowledge, a RAG system first retrieves relevant passages from a document collection and then uses them as context for answer generation.

This approach is useful for legal question answering because legal information must come from official documents. However, a standard RAG pipeline is not always sufficient for law. If documents are divided into simple text chunks, the system may lose information about the document structure, such as articles, clauses, points, appendices, and legal rank. As a result, the retrieved passages may be textually relevant but legally incomplete.

In this thesis, RAG is used as the main framework for grounding answers in Vietnamese labor-law documents. The system retrieves legal evidence before generating an answer, and the final response is expected to remain connected to the retrieved legal sources.

2.2 Legal Question Answering

Legal question answering has stricter requirements than general question answering because legal answers must be traceable to legal authority and context [11, 12]. In many domains, an answer may be considered useful if it is fluent and semantically relevant. In law, this is not enough. A legal answer must also show the legal basis that supports it.

This requirement is especially important in Vietnamese labor law. A practical question may involve several related documents, such as the Labor Code, a decree, a circular, or an appendix. These documents may explain, limit, or guide each other. Some rules may also be replaced or clarified by later documents. Therefore, a legal QA system must not only find similar text, but also preserve the legal context around each provision.

For this reason, the system in this thesis treats citations as a core part of the answer. The answer should indicate the relevant legal document and provision, so that users can inspect where the information comes from.

2.3 Hybrid Retrieval for Legal Evidence

Retrieval is the first important step in a RAG system. Dense retrieval methods are useful because they search by semantic meaning rather than exact lexical overlap [13, 14]. They can find passages that are related to a user question even when the wording is different. However, legal questions often contain exact references, such as article numbers, clause numbers, document names, or specific legal terms. These details may be missed by semantic search alone.

Sparse or keyword-based retrieval is useful for exact matching of legal references, document names, and terminology [15]. It can help find provisions that contain specific legal references or terms. For legal QA, both types of retrieval are useful: dense retrieval helps with meaning, while sparse retrieval helps with legal coordinates and exact wording.

This thesis therefore uses hybrid retrieval, combining semantic and lexical signals to support both meaning-based retrieval and exact legal-coordinate matching. The system combines semantic search with keyword-oriented search in Qdrant. This allows the retriever to handle both natural-language questions and questions that depend on specific legal references.

2.4 Graph-Augmented Retrieval

Legal documents are connected by structure and references. A law may contain general rules, while a decree provides implementation details. A circular may give forms, lists, or additional guidance. An appendix may contain the table or template needed to answer a specific question. These relationships are difficult to represent with a flat list of text chunks.

Graph-augmented retrieval addresses this problem by representing legal units and their relationships as a graph for retrieval support [16]. In this thesis, Neo4j is used to store relationships between documents, articles, clauses, points, appendices, topics, and related legal provisions [17]. The graph helps the system expand from an initially retrieved provision to other provisions that may also be needed.

This is important because legal relevance is not always the same as textual similarity. A provision may be necessary for the answer because it is a parent rule, an implementing rule, a cross-reference, or an appendix, even if it does not use the same words as the user question.

2.5 Evaluation Requirements for Legal QA

Evaluating a legal QA system requires more than checking whether the answer sounds correct. A generated answer may be fluent but still cite the wrong article, miss an important provision, or answer a question that is outside the available corpus. Therefore, evaluation should check both retrieval and answer grounding, following the broader motivation of automated RAG evaluation frameworks [18, 19].

This is also consistent with legal NLP benchmark design, where legal understanding and reasoning are evaluated beyond general text fluency [20].

In this thesis, retrieval is evaluated by whether the system finds the required legal citations and avoids distracting or forbidden citations. End-to-end answer quality is evaluated by checking whether the final answer is supported by retrieved evidence, whether its citations remain inside the retrieved context, and whether unsupported questions are refused or marked as insufficient context.

This evaluation design is intentionally deterministic. It focuses on reproducible software behavior rather than subjective judgment alone. Although this does not replace expert legal review, it provides a practical way to measure whether the system follows its main design goal: answering only when there is enough retrieved legal evidence.

2.6 Research Gap

Existing RAG systems show that external retrieval can improve the factual grounding of generated answers. Existing legal NLP research also shows that legal texts require careful treatment of context and citations. However, there remains a practical gap in building a deployed Vietnamese labor-law assistant that preserves legal hierarchy, retrieves related provisions, validates citations, and refuses unsupported questions within a scoped corpus.

This thesis addresses that gap by implementing a citation-grounded QA workflow for Vietnamese labor law. The focus is not on training a new language model, but on building a reliable software pipeline from legal corpus preparation to retrieval, graph-based evidence expansion, answer generation, citation validation, and benchmark evaluation.

3 System Design and Methodology

3.1 Overall RAG Pipeline

The system is designed as a citation-grounded Retrieval-Augmented Generation pipeline for Vietnamese labor-law question answering. Given a user question, the assistant retrieves relevant legal evidence from a scoped corpus, assembles the selected evidence, generates an answer, and checks whether the cited legal bases are supported by the retrieved context.

The pipeline is divided into two main stages. The first stage is the offline legal knowledge build. In this stage, selected legal documents are cleaned, structured, enriched with metadata, and indexed. The output of this stage is stored in two complementary forms: a Qdrant hybrid index for text retrieval [21] and a Neo4j legal graph for legal relationships.

The second stage is the online answering process. When a user submits a question, the system retrieves candidate provisions from Qdrant, expands related evidence through Neo4j when needed, merges and filters the results, and prepares a bounded context for answer generation. The generated answer is then

validated before being returned to the user.

Figure 1 summarizes this overall pipeline. It shows how legal documents are processed offline and how user questions are handled at runtime, from retrieval to citation validation.

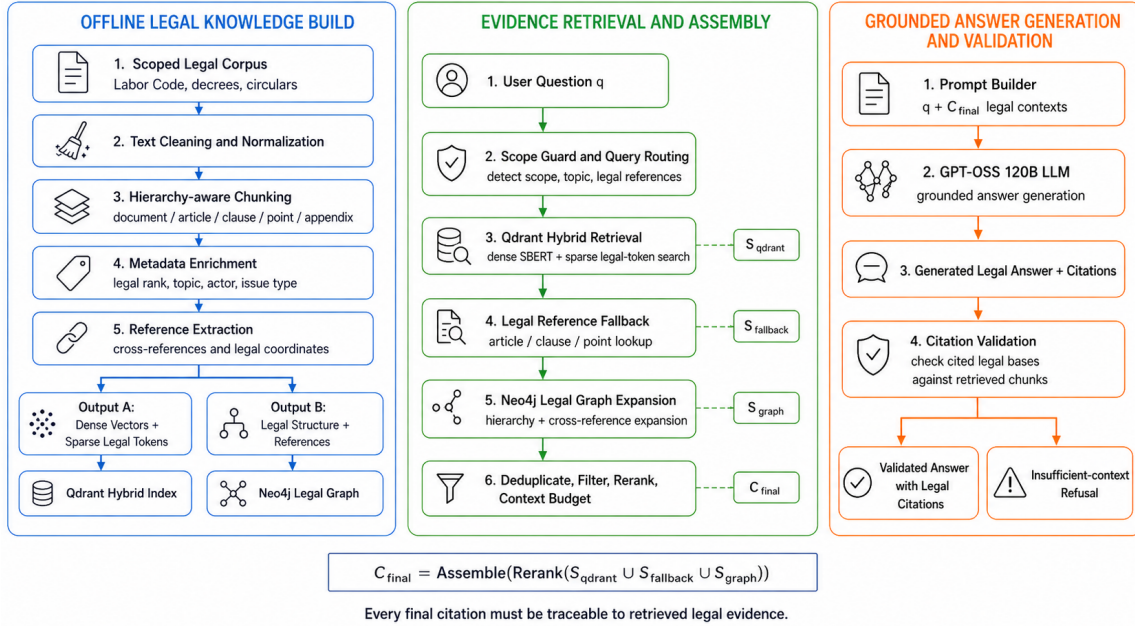


Figure 1. RAG pipeline for citation-grounded Vietnamese labor-law question answering.

3.2 Scoped Legal Corpus

The corpus is intentionally limited to a selected set of Vietnamese labor-law documents. This bounded scope makes the system easier to evaluate and also defines when the assistant should refuse a question or mark it as insufficient-context.

The final corpus contains six document groups: the Labor Code 2019 and the labor-related subset of the Civil Procedure Code 2015, Decree 135/2020/ND-CP and Decree 145/2020/ND-CP, and Circular 09/2020/TT-BLDTBXH and Circular 10/2020/TT-BLDTBXH. These documents were selected because they cover common labor-law issues such as employment contract termination, retirement age, minor workers, working time, and labor dispute procedures.

Vietnamese legal documents are often connected to each other through hierarchy, implementation guidance, and legal references. A code may define the general rule, a decree may provide implementation details, and a circular may contain forms, lists, or technical guidance. Some documents may also clarify, overlap with, or replace earlier rules. Therefore, the corpus cannot be treated as a flat collection of unrelated passages.

Table 1 summarizes the indexed legal corpus used in the system, based on the selected legal documents listed above.

Table 1. Scoped Legal Corpus Summary

Document group	Main role in the corpus	Rank	Chunks
Labor Code 2019	Core labor-law rules, including employment contracts, termination, working time, minor workers, and labor relations.	1	697
Civil Procedure Code 2015, labor-related subset	Labor dispute procedure and litigation-related provisions.	1	159
Decree 135/2020/ND-CP	Retirement age roadmap and related implementation details.	2	41
Decree 145/2020/ND-CP	Detailed guidance for implementing the Labor Code in labor conditions and labor relations.	2	520
Circular 09/2020/TT-BLDTBXH	Guidance related to minor workers and lists of permitted or restricted work.	3	73
Circular 10/2020/TT-BLDTBXH	Guidance and forms related to labor contract contents.	3	66
Total	Six indexed document groups		1,556

3.3 Hierarchy-Aware Legal Processing

The corpus processing pipeline is designed to preserve the legal structure of Vietnamese normative documents instead of splitting the documents into arbitrary fixed-length text windows. Each legal document is first normalized into a consistent textual format. During this step, the system removes irrelevant formatting noise, standardizes document titles, and separates the main content from headings, tables, appendices, and metadata.

After cleaning, each document is parsed according to its legal hierarchy. The main structural units include document, chapter, section, article, clause, point, and appendix. Each unit is then converted into an evidence chunk. A chunk may correspond to one clause, one group of points under the same clause, or one appendix-level unit, depending on the structure of the original legal text. This strategy keeps legal provisions close to their original legal coordinates while avoiding overly large contexts.

Each evidence chunk contains two groups of information. The first group is retrieval content, which includes the legal text used for semantic and lexical search. The second group is metadata, which records the legal source of the chunk, including document title, document type, article number, clause number, point label, appendix identifier, legal rank, topic labels, actor labels, and issue type. These metadata fields are important for filtering, graph construction, citation display, and citation validation.

Table 2 summarizes the main fields stored in each legal evidence chunk and their roles in the pipeline.

This representation allows later modules to retrieve evidence by both textual relevance and legal coordinates, while keeping each answer traceable to its original legal source.

This design allows the system to retrieve not only textually similar passages, but also legally identifiable evidence. For example, if a user question mentions a specific article, clause, or document name, the retriever can use the metadata to recover the corresponding legal coordinate. If a retrieved provision belongs to a lower-level document, the system can also inspect its relation to higher-level rules through the legal graph.

Table 2. Structure of a Legal Evidence Chunk

Component	Example fields	Purpose in the pipeline
Legal text	Provision text, shortened excerpt	Used as the main content for dense and sparse retrieval.
Legal hierarchy	Document, chapter, section, article, clause, point, appendix	Preserves the legal coordinates of each provision.
Citation metadata	Document title, article number, clause number, point label, appendix identifier	Supports citation display and citation validation after answer generation.
Legal rank	Law, decree, circular, national-level legal effect	Helps distinguish higher-level rules from implementation guidance.
Semantic labels	Topic, actor, issue type	Supports filtering, graph linking, and query-type-aware evidence selection.
Graph links	Parent relation, cross-reference, implementation guidance, appendix relation	Allows Neo4j expansion to recover legally connected evidence.

3.4 Hybrid Retrieval Pipeline

The first retrieval stage is implemented with Qdrant. The system uses a hybrid retrieval strategy that combines dense semantic retrieval and sparse lexical retrieval. Dense retrieval is used to capture semantic similarity between the user question and legal provisions, while sparse retrieval is used to preserve exact matching for article numbers, document names, legal terms, and Vietnamese legal expressions.

For each user question, the backend sends the normalized query to the retrieval module. The dense component retrieves provisions that are semantically close to the question, even when the user uses informal wording. The sparse component gives higher priority to exact legal markers such as “Điều 35”, “khoản 1”, “Bộ luật Lao động 2019”, or document identifiers such as “Nghị định 145/2020/ND-CP”. The two ranked lists are then merged into a single candidate list.

The system also applies reference recovery when the query explicitly contains a legal coordinate. This step is necessary because exact legal references are highly important in legal QA. If a user asks about a specific article or document, the system should not rely only on semantic similarity. It should directly recover the matching provision and include it in the candidate evidence pool when it exists in the indexed corpus.

The output of this stage is an initial set of candidate legal chunks. These candidates are not immediately sent to the generator. They are first passed to graph expansion, filtering, deduplication, and context budgeting so that the final context remains both legally complete and focused.

3.5 Policy-Controlled Legal Graph Expansion

The Neo4j legal graph is used after the initial Qdrant retrieval stage. Its purpose is to recover legally connected evidence that may not be ranked highly by text retrieval alone. The graph represents legal units as nodes and legal relationships as edges. The main node types include document, article, clause, point, appendix, topic, actor, and issue type. The main relationship types include hierarchy relationships, cross-reference relationships, implementation guidance relationships, topic relationships, and appendix-related relationships.

Graph expansion is policy-controlled. The system does not add all neighboring nodes of a retrieved provision. Instead, it decides whether expansion is needed based on the query type and the type of

relationship. For example, if a question requires implementation details, the system may expand from a Labor Code article to a related decree. If a question requires a table, form, or list, the system may expand toward an appendix or circular. If a question only asks for a direct definition, graph expansion can be limited to avoid adding distracting evidence.

This policy is important because legal relatedness does not always mean answer relevance. A provision may be connected to the same topic but still not be the correct legal basis for a specific question. Therefore, graph expansion is used as a controlled evidence recovery mechanism, not as an unrestricted neighborhood search.

The expanded evidence is merged with the Qdrant results. Duplicate chunks are removed, and each candidate is kept with its source information, retrieval score, graph relation type, and legal metadata. This makes it possible to inspect why a provision was selected and whether it came from direct retrieval or graph-based expansion.

3.6 Context Assembly and Budgeting

After retrieval and graph expansion, the system assembles the final legal context for answer generation. This step is necessary because the generator should receive enough evidence to answer the question, but not so much text that irrelevant provisions reduce answer quality.

The context assembly module first deduplicates overlapping chunks. It then reranks the candidates by combining retrieval relevance, legal-coordinate matching, graph-expansion relevance, legal rank, and query-type compatibility. Evidence that directly matches the user question or its legal coordinate is prioritized. Related evidence from graph expansion is included only when it helps complete the legal basis.

The final context is selected under a fixed budget. This budget limits the number of chunks and the amount of legal text passed to the answer generation module. When the question requires several connected provisions, the system may include a parent article, a related clause, or appendix-level evidence. When the question is simple, the system keeps only the most relevant provisions.

The purpose of context budgeting is to balance completeness and precision. A small context may miss necessary legal evidence, while a large context may introduce provisions that are related but not legally valid for the answer. The system therefore treats context assembly as a separate evidence selection step rather than directly passing all retrieved results to the generator.

3.7 Citation Validation and Insufficient-Context Handling

The answer generation module is required to answer only from the selected legal contexts. In the deployed system, a configured language-model provider generates a natural-language answer from the retrieved evidence. For reproducible evaluation, a deterministic extractive generation mode is used so that the benchmark result does not depend on sampling behavior or external model availability.

After generation, the citation validator checks whether the cited legal bases in the answer are present in the selected context. The validator detects three main problems: citations that do not appear in the retrieved evidence, article numbers that are unsupported by the context, and answers that refer to legal sources outside the indexed corpus. If any required citation is missing or unsupported, the answer is treated as invalid for evaluation.

The system also includes insufficient-context handling. If the retrieved evidence is not strong enough to support a legal answer, or if the question requires a document outside the indexed corpus, the assistant

should not provide a confident legal conclusion. Instead, it returns an insufficient-context response and explains that the indexed corpus does not contain enough legal basis for the question.

This mechanism is central to the safety of the system. The goal is not only to generate fluent legal explanations, but to ensure that each answer remains traceable to retrieved legal evidence. In this thesis, citation validation does not prove that every legal interpretation is perfect, but it provides a practical control layer against unsupported legal citations and out-of-scope answers.

4 Implementation and Deployment

4.1 Technology Stack

The system is implemented as a modular web application. Each component is responsible for a specific part of the legal question-answering pipeline, from corpus preparation to user interaction.

The backend is built with FastAPI [22]. It exposes the main chat endpoint, handles authenticated requests, connects to the retrieval modules, and returns answers with citations. Qdrant is used as the vector database for hybrid retrieval, while Neo4j stores the legal graph used for hierarchy and reference-based expansion. The frontend is developed with React, Vite, and TypeScript [23, 24]. Supabase provides user authentication, user profiles, conversation storage, message storage, and trace data [25].

For answer generation, the deployed system uses Groq as the configured language-model provider [26]. In the final reproducible evaluation, deterministic extractive generation is used instead of provider-based generation, so that the benchmark results do not depend on external model availability or sampling behavior.

Table 3 summarizes the main technologies and their roles in the system.

Table 3. Technology Stack and System Modules

Layer	Technology	System role
Corpus pipeline	Python	Validate, chunk, enrich, and export legal evidence.
Vector database	Qdrant	Store dense vectors, sparse retrieval signals, and payload metadata for hybrid retrieval.
Legal graph	Neo4j	Store legal hierarchy, references, topics, and graph expansion relationships.
Backend API	FastAPI	Provide chat, authentication-aware routes, retrieval integration, answer generation, and trace storage.
Frontend	React, Vite, TypeScript	Provide the user interface for questions, answers, citations, and conversation history.
Authentication and app data	Supabase	Manage authentication, user profiles, conversations, messages, and chat traces.
Answer generation	Configured language-model provider; deterministic extractive mode for evaluation	Generate grounded user-facing answers and support reproducible benchmark evaluation.

4.2 Offline Corpus, Index, and Graph Build

The offline build prepares the legal knowledge base before the system is used by end users. This stage starts from curated Vietnamese legal documents and converts them into structured evidence chunks.

Each chunk contains the legal text, citation information, document metadata, legal coordinates, and other retrieval-related fields. The system then builds two complementary representations from these chunks. The first representation is the Qdrant hybrid index, which stores dense vectors and sparse retrieval signals. The second representation is the Neo4j legal graph, which stores relationships between legal documents, articles, clauses, points, appendices, topics, and related provisions.

This offline build is important because it keeps expensive processing outside the online request path. It also ensures that the runtime system can retrieve legal evidence quickly while still preserving legal structure.

Figure 2 shows the deployment architecture and the main system components.

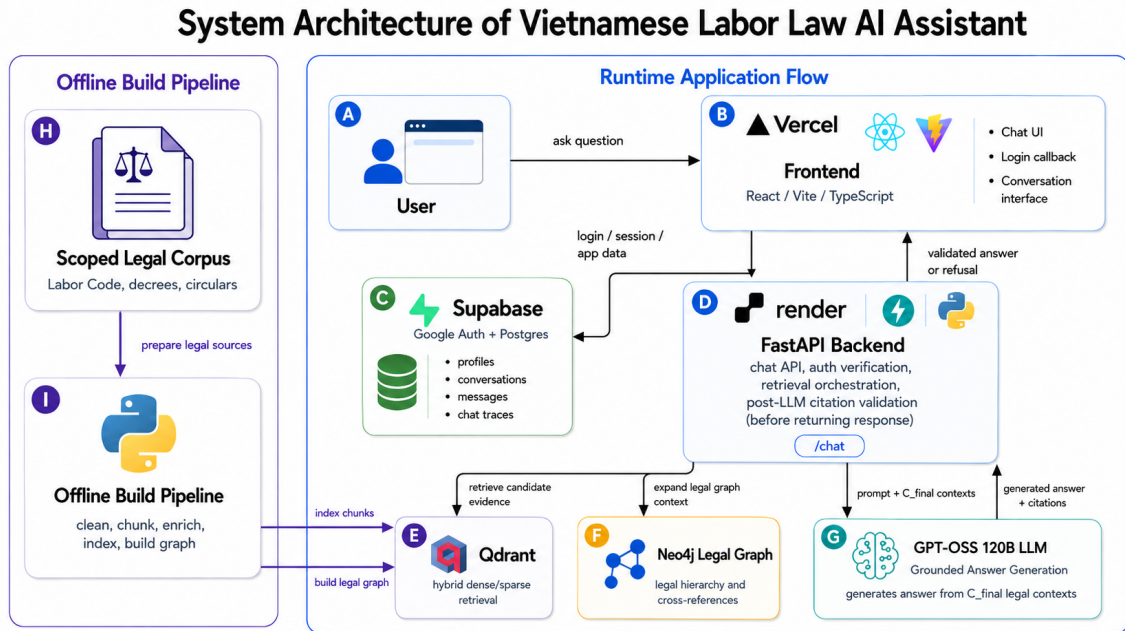


Figure 2. System architecture and deployment components of the Vietnamese Labor Law AI Assistant.

4.3 Backend Runtime and Chat Flow

The FastAPI backend is the main integration layer of the system. It receives user messages from the frontend, authenticates requests, retrieves legal evidence, calls the answer generation module, validates citations, and stores the conversation result.

The main chat flow starts when a user sends a question. The backend first checks the user session through Supabase authentication. It then sends the question to the retrieval pipeline. The retriever searches the Qdrant index, expands evidence through Neo4j when needed, and returns a set of selected legal contexts.

After retrieval, the backend passes the selected contexts to the answer generation module. If the system cannot find enough usable evidence, it returns an insufficient-context response instead of generating an unsupported legal answer. This behavior keeps answer generation dependent on retrieved legal sources.

The backend also stores useful trace information, such as the retrieved contexts, selected citations, la-

tency, and insufficient-context status. These traces help analyze system behavior during evaluation and debugging.

4.4 Answer Generation and Safety

The answer generation module is responsible for producing a response from the selected legal contexts. In the deployed application, Groq is used as the configured language-model provider for natural-language answers. The prompt instructs the model to answer only from the retrieved legal evidence and to include cited legal bases.

However, answer generation alone is not considered sufficient for a legal information system. After generation, the citation validator checks whether the cited legal bases actually appear in the retrieved evidence. It also detects unsupported article numbers and insufficient-context cases, which helps reduce hallucinated or unsupported legal references.

The system also includes scope control. Some questions require legal documents that are not included in the indexed corpus, such as regional minimum wage, insurance contribution rates, administrative penalties, foreign-worker work permits, trade-union fees, or personal income tax. When the system detects that the available corpus is insufficient, it should refuse or mark the answer as insufficient-context.

This design follows a conservative safety principle. It is better for the assistant to return a limited answer or refuse a question than to provide a fluent answer with unsupported legal citations.

4.5 Frontend and Deployment Architecture

The frontend is implemented with React, Vite, and TypeScript. It provides the user interface for asking questions, viewing answers, checking citations, and reviewing conversation history. The frontend communicates with the FastAPI backend through HTTP requests.

Authentication is handled through Supabase Auth, including Google OAuth. After login, the frontend sends the Supabase access token to the backend in the request header. This allows the backend to protect chat and conversation routes.

In deployment, the frontend can be hosted on Vercel, while the FastAPI backend can be hosted separately. Supabase stores user profiles, conversations, messages, and chat traces. Qdrant stores the retrieval index, and Neo4j stores the legal graph. This separation makes the system easier to maintain because each service has a clear role.

4.6 User Interface Demonstration

The user interface presents the assistant as a scoped Vietnamese labor-law information tool. The goal of the interface is not only to display an answer, but also to show the legal basis behind the answer.

Figure 3 shows an example of a supported in-corpus question. In this case, the assistant retrieves relevant legal evidence, generates an answer in Vietnamese, and displays the cited legal basis below the response.



Figure 3. Example in-corpus legal question answered with cited legal basis.

Figure 4 shows the opposite case. When the question is outside the indexed corpus or when the available evidence is insufficient, the assistant avoids giving an unsupported legal conclusion and returns an insufficient-context response.

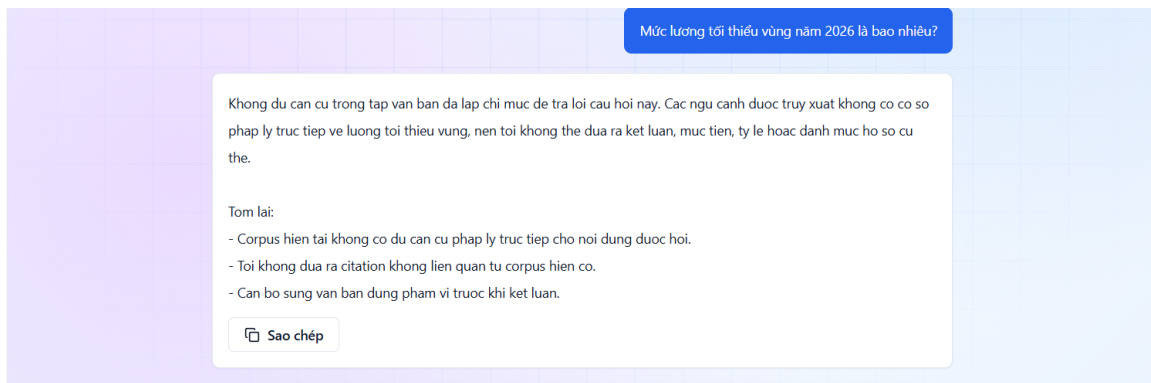


Figure 4. Insufficient-context response for an unsupported or out-of-corpus legal question.

These two examples demonstrate the two expected behaviors of the system: answering with citations when legal evidence is available, and refusing or marking insufficient context when the corpus does not support the question.

4.7 Evaluation Reproducibility

The implementation also includes a reproducible evaluation mode. This mode uses fixed benchmark files, a fixed index manifest, and structured output files. The purpose is to make the reported results repeatable.

The final evaluation does not depend on random language-model sampling. Instead, it uses deterministic extractive generation to evaluate whether the retrieval and citation-grounding pipeline works as expected. This choice makes the evaluation less fluent than a full LLM-based answer in some cases, but it provides more stable benchmark results.

This separation between deployed runtime generation and deterministic evaluation is important for RAG

systems whose results may otherwise vary with retrieval settings, prompts, or model sampling. The deployed system can use a configured language-model provider for user-facing answers, while the benchmark can remain reproducible for thesis evaluation.

5 Evaluation and Discussion

5.1 Benchmark Design and Evaluation Metrics

The system is evaluated on a fixed benchmark of 100 questions. The benchmark is designed to test the assistant as a complete legal question-answering system, not only as a text generator. It includes 94 in-corpus questions and 6 out-of-corpus questions. For in-corpus questions, the expected behavior is to retrieve the required legal basis and produce a grounded answer. For out-of-corpus questions, the expected behavior is to refuse the question or mark the answer as insufficient-context.

The benchmark covers different types of Vietnamese labor-law questions, including direct questions, definitions, document guidance, exceptions, multi-hop questions, table and appendix lookup, hard-negative citation cases, paraphrased user questions, dispute procedures, comparisons, scenarios, and unsupported topics.

The evaluation uses deterministic checks instead of subjective scoring alone, while following the broader goal of RAG evaluation: measuring retrieval quality, answer faithfulness, and grounding. This is important because a fluent legal answer may still be wrong if it cites the wrong article, misses a required legal basis, or answers a question outside the indexed corpus. Therefore, the evaluation focuses on retrieval quality, citation grounding, answer quality, and refusal behavior.

The benchmark design is also related to legal NLP evaluation efforts, which emphasize domain-specific reasoning and legal text understanding rather than general text fluency alone.

Table 4 summarizes the main evaluation metrics used in this thesis.

Table 4. Evaluation Metrics Summary

Metric	What it measures	Main interpretation
Recall@10	Whether required legal citations appear in the top retrieved contexts.	Higher values indicate better recovery of required evidence.
Required citation coverage	Whether the retriever finds the expected legal bases for in-corpus questions.	Measures evidence completeness before answer generation.
Forbidden citation violation rate	Whether distracting or invalid citations appear in retrieved results.	Lower values indicate cleaner retrieval and graph expansion.
Retrieval pass rate	Whether retrieval satisfies required and forbidden citation checks.	Shows whether the answer layer receives usable legal evidence.
Answer pass rate	Whether the generated answer satisfies deterministic answer-quality checks.	Measures answer adequacy after evidence selection.
Citation validation pass rate	Whether cited legal bases are supported by retrieved contexts.	Measures citation containment and grounding.
Adjusted end-to-end pass rate	Whether the full pipeline passes retrieval, answer, citation, and refusal checks.	Summarizes complete system behavior.
Out-of-corpus QA pass rate	Whether unsupported questions are refused or marked insufficient-context.	Measures conservative scope behavior.

Table 5 shows the composition of the 100-question benchmark.

Table 5. Benchmark Composition

Category	Queries
Direct QA	19
Definition QA	7
Document guidance QA	10
Exception-based QA	11
Multi-hop QA	7
Table and appendix lookup	10
Hard-negative citation QA	5
Paraphrased real-user QA	5
Labor dispute and procedure QA	5
Procedure QA	7
Comparison QA	4
Scenario-based QA	4
Out-of-corpus QA	6
Total	100

5.2 Retrieval Results

The first evaluation step measures whether the system can retrieve the required legal evidence. This is important because the final answer depends on the quality of retrieved contexts. If the correct legal provisions are not retrieved, the answer generation step may not have enough evidence to produce a valid

response.

Two retrieval settings are compared. The first setting uses hybrid retrieval only. In this setting, the system retrieves evidence from Qdrant using dense and sparse search signals. The second setting enables graph-augmented retrieval. In this setting, the system starts from Qdrant results and then expands related evidence through the Neo4j legal graph.

Table 6 reports the retrieval results.

Table 6. Retrieval and Graph Expansion Results

Mode	Queries	Recall@10	Coverage	Forbidden rate	Pass rate
Hybrid-only	94	0.735	0.735	0.032	0.649
Graph-augmented	94	0.835	0.835	0.043	0.766

The results show that graph-augmented retrieval improves Recall@10 from 0.735 to 0.835. The retrieval pass rate also increases from 0.649 to 0.766. This means that the legal graph helps the system recover relevant provisions that hybrid retrieval alone may miss.

However, graph expansion also slightly increases the forbidden citation violation rate, from 0.032 to 0.043. This shows an important trade-off. Legal graph connections can help recover missing evidence, but they can also introduce related provisions that are not valid for the specific question. Therefore, graph expansion must be controlled carefully and should not simply add every connected legal node.

5.3 End-to-End Results

The second evaluation step measures the full question-answering pipeline. This includes retrieval, context assembly, answer generation, citation validation, and refusal behavior.

Table 7 reports the final end-to-end results.

Table 7. End-to-End QA and Citation Validation Results

Metric	Value
Total benchmark queries	100
Adjusted end-to-end pass rate	0.780
Retrieval pass rate	0.780
Answer pass rate	0.980
Citation validation pass rate	1.000
Quality validation pass rate	0.980
Average final quality score	94.11
Low-information quotes	1
Unsupported article numbers	0
Unretrieved citations	0
Out-of-corpus QA pass rate	1.000
Graph expansion used	89 queries
Average graph depth	2.160

The final adjusted end-to-end pass rate is 0.780. The answer pass rate is 0.980, while the citation validation pass rate is 1.000. The out-of-corpus QA pass rate is also 1.000. These results show that the system is strong at keeping generated citations inside retrieved evidence and refusing unsupported questions.

The main limitation is retrieval. The retrieval pass rate is also 0.780, which indicates that most end-to-end failures come from missing or incomplete retrieved evidence rather than from citation hallucination after generation. In other words, once the system has the correct legal context, the answer and citation validation components usually behave reliably.

5.4 Category-Level Results

The benchmark results are not uniform across all question types. The system performs well on direct questions, definition questions, document guidance questions, exception-based questions, comparison questions, procedure questions, and scenario-based questions. These categories usually require legal evidence that can be retrieved directly or with limited expansion.

The weaker categories are labor-dispute procedure questions, hard-negative citation questions, paraphrased real-user questions, multi-hop questions, and table or appendix lookup. These failures are expected because these categories require more complex evidence selection. For example, table lookup may require retrieving a specific appendix or table row. A paraphrased question may hide the legal concept behind informal wording. A hard-negative case may contain provisions that are legally related but not actually valid for the answer.

This pattern shows that graph expansion improves retrieval, but it does not solve every retrieval problem. The system still needs stronger query understanding, better table-aware retrieval, and more precise filtering of legally related but distracting provisions.

5.5 Case Studies and Error Taxonomy

To better understand the system behavior, representative successful and failed cases are analyzed. Successful cases show that the system can answer correctly when the required legal basis is retrieved. For example, retirement-age questions can combine the Labor Code with Decree 135/2020/ND-CP table evidence. Out-of-corpus questions, such as minimum wage by region, are refused because the relevant legal instrument is not included in the indexed corpus.

Failed cases show where the current system remains limited. Some paraphrased questions fail because the system does not map the user’s wording to the correct legal concept. Some table or appendix questions fail because the required evidence is stored in a more structured form than ordinary text. Some hard-negative cases fail because the graph adds legally related provisions that are close to the topic but not correct for the specific answer.

Table 8 summarizes representative cases and error types.

Table 8. Case Studies and Error Taxonomy

Case or error	Result	Interpretation
Female retirement age in 2026	Pass	Retrieved Labor Code and Decree 135 table evidence.
Minimum wage by region in 2026	Pass refusal	Correctly identified insufficient indexed context.
Temporary transfer paraphrase	Fail	Missed Labor Code Article 29 required citations.
Domestic worker contract template	Fail	Missed appendix/form evidence and produced low-information answer.
Holiday overtime pay hard negative	Fail	Mixed missing citation with over-expansion to overtime limit provisions.
Labor-dispute court choice	Fail	Cross-code procedure retrieval remained incomplete.
Retrieval missing required context	20 cases	Dominant failure reason.
Retrieval over-expansion	4 cases	Related but forbidden citations entered top results.
Low-information answer	2 cases	Extractive answer lacked enough useful rule content.
Answer missing required rule	1 case	Answer quality failed after retrieval gap.

The dominant failure reason is missing required context. This confirms that the main bottleneck is still retrieval and evidence selection. Citation validation is reliable in the final run, but it cannot recover a legal basis that was not retrieved in the first place.

5.6 Discussion

The evaluation supports the main design choice of this thesis. Legal question answering benefits from combining text retrieval with legal structure. Hybrid retrieval is useful for finding semantically and lexically relevant provisions, while graph expansion helps recover related legal evidence through hierarchy, cross-references, and implementation relationships.

The improvement from 0.735 to 0.835 in Recall@10 shows that Neo4j contributes useful evidence beyond flat retrieval. This is especially important in Vietnamese labor law, where a complete answer may depend on several connected documents. A Labor Code provision may need a decree for implementation details, or a circular or appendix may provide the practical information needed by the user.

At the same time, the results show that graph expansion must be controlled. The increase in forbidden citation violations means that legal relatedness is not always the same as answer relevance. A neighboring provision in the graph may be connected to the topic, but still not be the correct legal basis for the question. This is why the system uses policy-controlled expansion, filtering, reranking, and context budgeting.

The strongest result of the system is citation containment. The final citation validation pass rate is 1.000, with no unretrieved citations and no unsupported article numbers in the final run. This does not prove that every legal interpretation is perfect, but it shows that the system can keep its citations within retrieved evidence. For a scoped legal information assistant, this is an important safety property.

The main remaining weakness is retrieval robustness. Informal user questions, table-heavy answers, hard-negative cases, and cross-document procedure questions still create errors. Future improvement should therefore focus less on making the language model more fluent and more on improving query rewriting, table retrieval, graph filtering, and legal evidence selection.

5.7 Threats to Validity

The evaluation has several limitations. First, the benchmark is manually constructed and contains 100 questions. Although it covers several important question types, it cannot represent all possible Vietnamese labor-law questions.

Second, the system is evaluated within a scoped corpus of selected legal documents. The results should not be interpreted as coverage of all Vietnamese law. Questions that require documents outside this corpus may still be unsupported.

Third, deterministic validation is useful for reproducibility, but it cannot fully replace expert legal review. It can check whether citations are retrieved and whether some answer rules are satisfied, but it cannot judge every possible issue of legal interpretation.

Fourth, the final reproducible evaluation uses deterministic extractive generation. This makes the results stable, but it may produce answers that are less fluent than the deployed language-model path. Therefore, the evaluation should be understood as a test of the citation-grounded pipeline rather than a complete measurement of user-facing answer quality.

Finally, the results depend on the fixed benchmark, index manifest, and evaluation outputs used in this thesis. Future work should extend the benchmark with more real user questions, expert-reviewed answers, temporal validity checks, and a larger versioned legal corpus.

6 Conclusion and Future Work

6.1 Conclusion

This thesis presented a citation-grounded AI assistant for Vietnamese labor-law question answering. The system is built around a scoped legal corpus and is designed to retrieve legal evidence before generating an answer. Its main purpose is to help users access labor-law information with traceable legal bases, rather than to provide unrestricted legal advice.

The implemented system combines several components into one complete workflow. It processes selected Vietnamese labor-law documents into structured evidence chunks, stores them in a Qdrant hybrid index, represents legal relationships in a Neo4j graph, retrieves and assembles relevant contexts, generates grounded answers, and validates citations before returning the final response. The system is also deployed as a web application with a FastAPI backend, a React/Vite frontend, Supabase authentication and data storage, and trace logging for evaluation.

The evaluation shows that graph-augmented retrieval improves the system’s ability to recover relevant legal evidence. Recall@10 increases from 0.735 with hybrid retrieval only to 0.835 when graph expansion is enabled. The final end-to-end pass rate is 0.780. The citation validation pass rate and the out-of-corpus refusal pass rate both reach 1.000, showing that the system is strong at keeping citations inside retrieved evidence and refusing unsupported questions within the benchmark.

However, the results also show clear limitations. Most failures come from retrieval gaps rather than answer generation. The system still struggles with paraphrased user questions, table and appendix lookup, hard-negative citation cases, and some cross-document procedure questions. These limitations confirm that legal question answering depends not only on the language model, but also on the quality of legal evidence retrieval and selection.

Overall, this thesis shows that a reliable legal QA assistant should be treated as a full software pipeline. For Vietnamese labor-law questions, preserving legal structure, retrieving related provisions, controlling graph expansion, and validating citations are all important for producing answers that users can inspect and trust.

6.2 Future Work

Future work should first improve retrieval robustness. Better query rewriting could help the system understand informal user questions and map them to the correct legal concepts. Table-aware and appendix-aware retrieval should also be improved, especially for questions that require retirement-age tables, forms, lists, or structured legal information.

Second, graph expansion should be refined. The current results show that Neo4j helps recover missing evidence, but it can also introduce related yet distracting provisions. Future versions should improve graph filtering, query-type classification, and ranking rules so that graph expansion adds only legally useful evidence for each question.

Third, the legal corpus should be expanded and versioned. The current system is limited to selected Vietnamese labor-law documents. A larger corpus should include additional decrees, circulars, amendments, official guidance, and documents related to wages, insurance, administrative penalties, foreign workers, taxation, and trade unions. Versioning is also important because legal documents may be amended, replaced, or become outdated over time.

Fourth, the evaluation should include expert legal review. Deterministic benchmark checks are useful for reproducible software evaluation, but they cannot fully judge legal interpretation. Future evaluation should combine automatic citation checks with expert-reviewed answers, real anonymized user questions, and temporal-validity checks.

Finally, the user-facing application can be improved with clearer citation display, better explanation of insufficient-context cases, and more transparent evidence inspection. These improvements would make the assistant easier to use for non-specialist users while keeping the system bounded to verified legal sources.

6.3 Final Remark

The main lesson of this thesis is that legal question answering is not only a text generation problem. A fluent answer is not enough if the legal basis cannot be verified. For a Vietnamese labor-law assistant, reliability depends on the whole pipeline: structured legal corpus preparation, hybrid retrieval, graph-based evidence expansion, careful context selection, and strict citation validation.

References

- [1] National Assembly of Vietnam. *Law No. 80/2015/QH13 on Promulgation of Legislative Documents*. 2015.
- [2] National Assembly of Vietnam. *Law No. 63/2020/QH14 Amending and Supplementing the Law on Promulgation of Legal Normative Documents*. 2020.
- [3] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38.
- [4] National Assembly of Vietnam. *Labor Code No. 45/2019/QH14*. 2019.
- [5] National Assembly of Vietnam. *Civil Procedure Code No. 92/2015/QH13*. 2015.
- [6] Government of Vietnam. *Decree No. 135/2020/ND-CP on Retirement Age*. 2020.
- [7] Government of Vietnam. *Decree No. 145/2020/ND-CP Providing Guidance on Working Conditions and Labor Relations under the Labor Code*. 2020.
- [8] Ministry of Labour, Invalids and Social Affairs of Vietnam. *Circular No. 09/2020/TT-BLDTBXH Providing Guidance on Minor Workers under the Labor Code*. 2020.
- [9] Ministry of Labour, Invalids and Social Affairs of Vietnam. *Circular No. 10/2020/TT-BLDTBXH Providing Guidance on Employment Contract Contents and Related Labor-Code Matters*. 2020.
- [10] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. 2020.
- [11] Neel Guha et al. “LegalBench: A Collaboratively Curated Benchmark Dataset for Measuring Legal Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2308.11462* (2023).
- [12] Haoxi Zhong et al. “Iterative Contextualization for Legal Question Answering”. In: *arXiv preprint arXiv:2009.03846* (2020).
- [13] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 2020.
- [14] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019.
- [15] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
- [16] Darren Edge et al. “GraphRAG: Unlocking LLM Discovery on Narrative Private Data”. In: *arXiv preprint arXiv:2404.16130* (2024).
- [17] Neo4j, Inc. *Neo4j Documentation*. 2026. URL: <https://neo4j.com/docs/>.
- [18] Shahul Es et al. “Ragas: Automated Evaluation of Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2309.15217* (2023).
- [19] Jon Saad-Falcon et al. “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems”. In: *arXiv preprint arXiv:2311.09476* (2023).

- [20] Ilias Chalkidis et al. “LexGLUE: A Benchmark Dataset for Legal Language Understanding in English”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 2022.
- [21] Qdrant Team. *Qdrant Vector Search Engine Documentation*. 2026. URL: <https://qdrant.tech/documentation/>.
- [22] FastAPI Contributors. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/>.
- [23] Meta Open Source. *React Documentation*. 2026. URL: <https://react.dev/>.
- [24] Vite Team. *Vite Documentation*. 2026. URL: <https://vite.dev/guide/>.
- [25] Supabase. *Supabase Documentation*. 2026. URL: <https://supabase.com/docs>.
- [26] Groq. *Groq API Documentation*. 2026. URL: <https://console.groq.com/docs/overview>.

A Example Legal Evidence Chunk

The example chunk below shows one representative evidence unit produced by the corpus pipeline. The legal text is shortened for presentation, but the metadata fields are kept to show how the system stores a retrievable legal evidence unit.

```
{
  "chunk_id": "45_2019_QH14_Dieu_35_Khoan_1_Diem_a_b_c_d",
  "document": {
    "document_id": "45-2019-qh14",
    "document_title": "Bộ luật Lao động 2019",
    "document_type": "bo_luat",
    "issuing_authority": "Quốc hội",
    "status": "in_effect"
  },
  "legal_hierarchy": {
    "chapter": "Chương III. HỢP ĐỒNG LAO ĐỘNG",
    "section": "Mục 3. CHẤM DỨT HỢP ĐỒNG LAO ĐỘNG",
    "article": "Điều 35",
    "article_title": "Quyền đơn phương chấm dứt hợp đồng lao động của người lao động",
    "clause": "khoản 1",
    "points": ["a", "b", "c", "d"]
  },
  "legal_rank": {
    "normative_rank": 1,
    "rank_label": "highest",
    "effective_level": "national"
  },
  "topic": [
    "bao_truoc",
    "cham_dut_hop_dong_lao_dong",
    "don_phuong_cham_dut",
    "hop_dong_lao_dong"
  ],
  "actor": [
    "nguoi_lao_dong",
    "nguoi_su_dung_lao_dong"
  ],
  "issue_type": [
    "can_cu_cham_dut",
    "quyen_don_phuong_cham_dut",
    "thoi_han_bao_truoc",
    "loai_hop_dong",
    "don_phuong_cham_dut"
  ],
  "evidence_text_excerpt": "1. Người lao động có quyền đơn phương chấm dứt hợp đồng lao
  ⇨ động nhưng phải báo trước [...] a) Ít nhất 45 ngày nếu làm việc theo hợp đồng lao
  ⇨ động không xác định thời hạn; b) Ít nhất 30 ngày nếu làm việc theo hợp đồng lao động
  ⇨ xác định thời hạn từ 12 tháng đến 36 tháng; [...] d) Đối với một số ngành, nghề, công
  ⇨ việc đặc thù thì thời hạn báo trước được thực hiện theo quy định của Chính phủ.",
  "citation_label": "Bộ luật Lao động 2019, Điều 35, khoản 1, các điểm a, b, c, d"
}
```

B Example Legal Graph

Figure 5 shows the retirement-age topic as a concrete graph example. A complete answer may require moving from the Labor Code provision to the implementing decree and then to the appendix table that contains year-specific retirement ages.

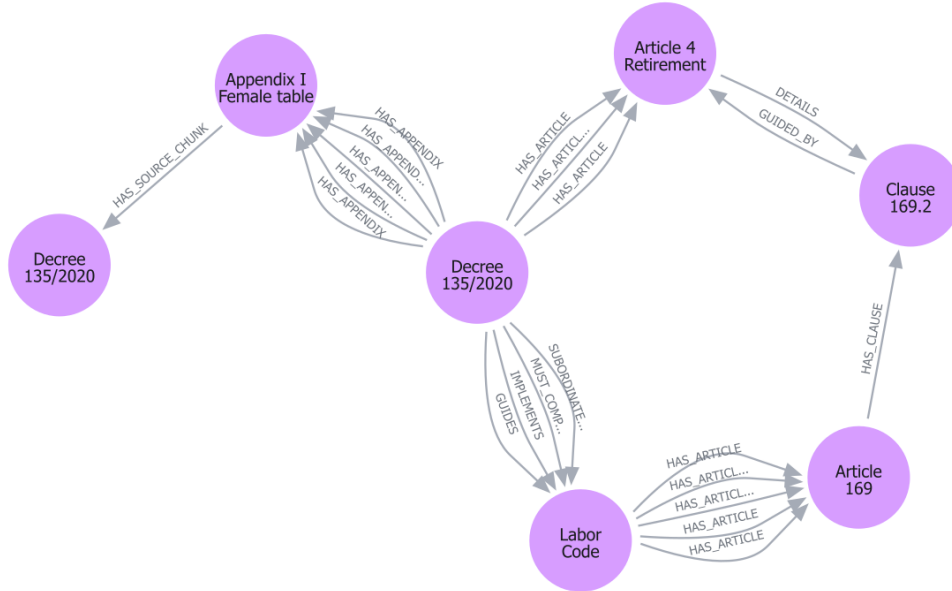


Figure 5. Example legal graph expansion for retirement-age evidence.

The graph uses the actual implementation labels and relationship types shown below.

Labels:

Legal_Document, Legal_Article, Legal_Clause,
Legal_Appendix, Evidence_Chunk

Relationships:

HAS_ARTICLE, HAS_CLAUSE, HAS_APPENDIX, HAS_SOURCE_CHUNK,
DETAILS, GUIDED_BY, IMPLEMENTS, GUIDES, MUST_COMPLY_WITH,
SUBORDINATE_TO

C Benchmark Design Details

The benchmark contains 100 queries. Each item stores the question, category, topic, expected documents, required citations, optional forbidden citations, expected answer points, difficulty, and whether the item is expected to benefit from graph or normative-hierarchy retrieval. Table 9 explains the design role of each category.

Table 9. Benchmark category design.

Category	What it tests	Example question type	Expected system behavior
Definition QA	Legal definitions.	“What is an employee under the Labor Code?”	Retrieve the exact definition article or clause.
Direct QA	Single-provision legal lookup.	“How much notice must an employee give before unilateral termination?”	Return the direct provision and cite it.
Document guidance QA	Connection between primary law and implementing guidance.	“Which circular guides labor-contract content?”	Retrieve both the Labor Code basis and the lower-rank guidance.
Exception-based QA	Exceptions, prohibitions, or special cases.	“When may an employee quit without notice?”	Identify the exception rule and avoid unrelated general rules.
Procedure QA	Legal procedure, deadlines, and obligations.	“What must the employer do after probation ends?”	Retrieve the procedure rule and summarize the required action.
Scenario-based QA	Mapping a user fact pattern to legal provisions.	“What happens if the employee unlawfully terminates the contract?”	Select the legally relevant rule for the described situation.
Comparison QA	Comparing two legal regimes.	“Compare severance allowance and job-loss allowance.”	Retrieve evidence for both sides of the comparison.
Multi-hop QA	Questions requiring several linked provisions.	“How is annual leave calculated for a partial year and does probation count?”	Combine multiple provisions without dropping required citations.
Calculation or table lookup	Tables, appendices, and numeric lookup.	“Female retirement age in 2026?”	Retrieve article evidence plus the relevant appendix or table chunk.
Hard-negative citation QA	Avoiding close but wrong legal bases.	“Holiday overtime pay, not overtime-hour limits.”	Retrieve the required citation and keep forbidden citations out of the top results.
Paraphrased real-user QA	Informal user wording.	“My company moved me to another job for almost three months; is that okay?”	Map informal phrasing to the correct legal concept.
Labor dispute procedure QA	Labor dispute procedure and civil procedure links.	“Can I sue over unlawful termination and which court basis applies?”	Retrieve both labor-law and BLTTDS procedural evidence.
Out-of-corpus QA	Unsupported topics outside the indexed corpus.	“What are the 2026 regional minimum wages?”	Refuse or return insufficient context rather than guessing.

The category distribution is intentionally mixed. Easy definition and direct questions test basic retrieval, while multi-hop, appendix, paraphrase, hard-negative, and dispute-procedure questions stress the parts of the system where legal QA is most likely to fail.

D Additional Evaluation Examples

Table 10 summarizes representative cases from the final 100-query evaluation. The examples are selected to show both successful behavior and the main remaining retrieval weaknesses.

Table 10. Representative final evaluation cases.

Case Type	Tested Capability	Outcome	Interpretation
Simple pass	Employee notice period for unilateral termination	Pass. The system retrieved and cited Labor Code Article 35.	Direct lookup works when the question closely matches the legal issue.
Multi-source pass	Female retirement age in 2026	Pass. The system combined the Labor Code, Decree 135, and the retirement-age table.	Graph/table expansion helps when the answer requires connected legal sources.
Correct refusal	Regional minimum wage in 2026	Pass refusal. The system returned insufficient context because the wage instrument was outside the corpus.	Scope control prevents unsupported legal claims.
Paraphrase failure	Temporary transfer described in informal wording	Fail. The system missed the required Labor Code Article 29 evidence.	Informal user wording remains difficult for retrieval.
Appendix/table failure	Domestic-worker contract form in Decree 145	Fail. The system missed the required form/appendix evidence.	Forms and appendices need stronger table-aware retrieval.
Hard-negative failure	Holiday overtime pay confused with overtime limits	Fail. Retrieval mixed pay rules with hour-limit provisions.	Legal relatedness is not always answer relevance.